Week 1 Day-by-Day Plan: PyTorch + Hugging Face + Multilingual NLP

Windows + VS Code Local Development

> **Goal**: By end of Week 1, you have a working toxicity classification function tested across English, Hindi, and Arabic with documented benchmarks.

Pre-Setup (30 min — Do this FIRST)

Open **PowerShell** and run:

```powershell
cd C:\Users\YourUsername\Projects
mkdir multilingual-content-moderator
cd multilingual-content-moderator
python -m venv venv
.\venv\Scripts\Activate

# Install core dependencies
python -m pip install --upgrade pip
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
pip install transformers accelerate langdetect
pip install pandas matplotlib plotly numpy
pip install loguru tqdm
pip install fastapi uvicorn pydantic pydantic-settings python-multipart
pip install pytest httpx black isort flake8
pip install streamlit

# Verify
python -c "import torch; print(f'PyTorch {torch.__version__}')"
python -c "from transformers import pipeline; print('HuggingFace OK')"

# Open VS Code
code .
```

Then unzip the project files into this folder and configure VS Code:

- `Ctrl+Shift+P` → "Python: Select Interpreter" → pick `.\venv\Scripts\python.exe`
- Install extensions: **Python**, **Pylance**, **GitLens**

Git + GitHub Setup (Do this right after unzipping files)

```powershell
# Inside your project folder, with venv activated:

# Initialize git
git init

# Create repo on GitHub:
# 1. Go to github.com/new
# 2. Name: multilingual-content-moderator
# 3. Public, NO readme, NO gitignore (you have both already)
# 4. Click Create

# Connect and push
git add .
git commit -m "Initial project setup: skeleton, notebooks, Week 1 plan"
git remote add origin https://github.com/SonaliBalekundri/multilingual-content-moder
git branch -M main
git push -u origin main
```

See `Git_GitHub_Guide.md` for detailed instructions and troubleshooting.

How to Run Notebook Files in VS Code

The `.py` files in `notebooks/` use `# %%` cell markers. VS Code treats each `# %%` as a runnable cell:

1. Open any `.py` file in `notebooks/`
2. You'll see **"Run Cell"** buttons above each `# %%` marker
3. Click **Run Cell** or press **Shift+Enter** to run that cell
4. Output appears in an **Interactive Window** panel on the right
5. Work through cells one at a time, top to bottom

Day 1 (Monday) — PyTorch Fundamentals Part 1 (2-3 hours)

File: `notebooks/01_pytorch_basics.py`

What to do

1. Open `notebooks/01_pytorch_basics.py` in VS Code
2. Work through **Sections 1-3** (Tensors, Operations, Autograd)
3. Click "Run Cell" or press Shift+Enter for each `# %%` block

4. Do ALL the exercises marked with ✏️

Key concepts to nail today

- Tensor shapes and what each dimension means in NLP
- **Sigmoid vs Softmax** — why sigmoid for multi-label
- `torch.no_grad()` — why and when to use it

Done when

- [ ] Can create tensors of any shape
- [ ] Can explain sigmoid vs softmax in your own words
- [ ] Completed the exercises in Sections 1-3
- [ ] `git add . && git commit -m "Day 1: PyTorch basics - tensors, ops, autograd"`

Day 2 (Tuesday) — PyTorch Fundamentals Part 2 (2-3 hours)

File: `notebooks/01_pytorch_basics.py`

What to do

1. Work through **Sections 4-5** (nn.Module, Device Management)
2. Modify `SimpleToxicityClassifier` — try changing layer sizes
3. Understand what happens during a forward pass

Done when

- [ ] Can build a simple nn.Module from scratch
- [ ] Understand the full forward pass: input → model → output
- [ ] `git commit -m "Day 2: nn.Module, device management complete"`

Day 3 (Wednesday) — Hugging Face Transformers (2-3 hours)

File: `notebooks/02_huggingface_intro.py`

What to do

1. Open `notebooks/02_huggingface_intro.py`
2. Work through all 5 sections
3. **Important:** First model download is ~1.1GB (2-5 min on decent internet)
4. After first download it's cached in `C:\Users\YourUsername\.cache\huggingface\`

Low RAM note

If your system struggles, change the model name at the top of the file:

```python
MODEL_NAME = "citizenlab/distilbert-base-multilingual-cased-toxicity"
```

Done when

- [ ] Tokenized text in English, Hindi, and Arabic
- [ ] Loaded the toxicity model successfully
- [ ] Ran inference using both pipeline and manual approach
- [ ] `git commit -m "Day 3: HuggingFace intro, model loaded, inference working"`

Day 4 (Thursday) — Toxicity Models Deep Dive (2-3 hours)

File: `notebooks/03_toxicity_models.py`

What to do

1. Build the production-ready `classify()` function
2. Experiment with thresholds (0.3 to 0.8)
3. Test across all three languages — note differences
4. Test edge cases (sarcasm, short text, mixed language)

Done when

- [ ] `classify()` works reliably with threshold parameter
- [ ] Tested threshold values from 0.3 to 0.8
- [ ] Identified at least 2 edge cases where the model struggles
- [ ] `git commit -m "Day 4: Production classify function, threshold experiments"`

Day 5 (Friday) — Batch Testing + Refinement (2-3 hours)

File: `notebooks/03_toxicity_models.py`

What to do

1. Run batch processing test (20 texts)
2. Add 10+ edge cases and test them
3. Refine your classify() function
4. Review `app/models/moderator.py` — compare with your function

Done when

- [ ] Batch processing works on 20+ texts
- [ ] Tested 10+ edge cases and documented results
- [ ] Know which threshold value you'll recommend as default
- [ ] `git commit -m "Day 5: Batch processing, edge cases, threshold tuning"`

Day 6 (Saturday) — Benchmarking (3-4 hours)

File: `notebooks/04_multilingual_benchmark.py`

What to do — this is the BIG day

1. Expand benchmark dataset to 20+ per language (60+ total)
2. Run the full benchmark
3. Calculate per-language accuracy, precision, recall, F1
4. Analyse errors — which texts does the model get wrong?
5. Generate charts — they save automatically to `docs/`

Done when

- [ ] Benchmark dataset has 60+ labelled examples
- [ ] Metrics computed per language
- [ ] Charts saved to `docs/benchmark_results.png` and `docs/score_distribution.png`
- [ ] Error analysis documented
- [ ] `git commit -m "Day 6: Full multilingual benchmark with charts"`

Day 7 (Sunday) — Documentation + Week 1 Wrap (1-2 hours)

What to do

1. Update `README.md` with actual benchmark results
2. Replace TBD values with your numbers
3. Add chart images: `![Benchmarks](docs/benchmark_results.png)`
4. Write a LinkedIn post about what you built this week

LinkedIn post template

```
Week 1 of my AI/ML journey: Built a multilingual toxicity classifier! 🛡️
```

```
What I did:
- Loaded XLM-RoBERTa from Hugging Face
- Tested toxicity detection across English, Hindi, and Arabic
- Benchmarked performance: [your numbers here]

Key insight: [something surprising you found]

Next week: Wrapping this in a FastAPI service with batch processing.

#MachineLearning #NLP #AIEngineering #BuildInPublic #HuggingFace
```

Done when

- [ ] README has real benchmark numbers and charts
- [ ] LinkedIn post published
- [ ] `git add . && git commit -m "Day 7: README updated, Week 1 complete" && git push`

Daily Workflow Reminder

Every time you sit down to code:

```powershell
cd C:\Users\YourUsername\Projects\multilingual-content-moderator
.\venv\Scripts\Activate
code .
```

Week 1 Summary

Day	Focus	Hours	Key Output
1	PyTorch: tensors, ops, autograd	2-3h	Core PyTorch concepts
2	PyTorch: nn.Module, devices	2-3h	Build and run neural networks
3	Hugging Face: tokenizers, models	2-3h	Toxicity model loaded and running
4	Toxicity models: thresholds	2-3h	Production-ready classify() function
5	Batch processing, edge cases	2-3h	Robust classification
6	Benchmarking across languages	3-4h	Full benchmark with metrics and charts
7	Documentation, LinkedIn	1-2h	Updated README, public post

Day	Focus	Hours	Key Output
Total		**15-20h**	**Working classifier with benchmarks**